

1)

```
#include <stdio.h>
struct aluno{
    int n_matric;
    char nome[20];
    int n1,n2,n3;
}pessoal;
int main(){
    printf("o tamanho da estrutura é %d bytes ", sizeof(pessoal));
    return 0;
}
```

2)

```
#include <stdio.h>
#include <stdlib.h>

typedef struct{
    char nome[30];
    int idade;
    char ender[80];
}cadastro;

cadastro* alocar(int n){
    cadastro* pessoa=(cadastro*)malloc(n*sizeof(cadastro));
    if(pessoa==NULL){
        printf("ERRO NA ALOCACAO DA MEMORIA!");
        exit(1);
    }
    return pessoa;
}

int main(){
    int n;
    cadastro* pessoa;
    printf("quantas pessoas para ser cadastrado?\n");
    scanf("%d",&n);
    pessoa=alocar(n);
    for(int i=0; i<n; i++){
        printf("digite o nome para a pessoa %d", i+1);
        setbuf(stdin, NULL);
        fgets(pessoa[i].nome, 30, stdin);
        printf("digite a idade da pessoa %d", i+1);
        scanf("%d", &pessoa[i].idade);
    }
}
```

```

        printf("digite o endereco da pessoa %d", i+1);
        setbuf(stdin, NULL);
        fgets(pessoa[i].ender, 80, stdin);
    }
    free(pessoa);
    return 0;
}

```

3)

```

#include<stdio.h>
#include<stdlib.h>

int main(){
    int tamanho;
    int *vetor;
    printf("digite o tamanho do vetor que voce queira:\n");
    scanf("%d", &tamanho);
    vetor=(int *)malloc(tamanho*sizeof(int));
    if(vetor == NULL){
        printf("deu erro!");
        exit(1);
    }
    for (int i=0; i<tamanho; i++){
        printf("digite o valor no vetor %d",i+1);
        scanf("%d", &vetor[i]);
    }
    for(int i=0; i<tamanho; i++){
        printf("%d\n", vetor[i]);
    }
    free(vetor);
    return 0;
}

```

4)

```

#include<stdio.h>
#include<stdlib.h>

int verificar(int **p, int l, int c, int v){
    for(int i = 0; i < l; i++){

```

```

        for(int j = 0; j < c; j++){

            if(p[i][j] == v){
                return 1;
            }
        }
    }
    return 0;
}

void preencher(int **p, int l, int c){
    for(int i=0; i<l; i++){
        for (int m=0;m<c;m++ ){
            printf("digite o valor para linha %d coluna %d", i, m);
            scanf("%d", &p[i][m]);
        }
    }
}

int main(){
    int **matriz, l, c,n, s;
    do{
        printf("digite a quantidade de linhas:\n");
        scanf("%d", &l);
        printf("digite a quantidade de colunas:\n");
        scanf("%d", &c);
    }while(l<0||c<0);

    matriz=(int **)malloc(l*sizeof(int*));
    if(matriz==NULL){
        printf("deu erro na alocação da memória de linhas da matriz");
    }
    for (int i=0; i<l; i++){
        matriz[i]=(int *)malloc(c*sizeof(int));
        if(matriz[i]==NULL){
            printf("deu erro na alocação da memória de coluna da matriz");
        }
    }

    preencher(matriz, l,c);
    printf("digite um valor:\n");

```

```

scanf("%d", &n);
s=verificar(matriz, l, c,n );
printf("retorna : %d",s);
for (int i=0; i<l; i++){
    free(matriz[i]);
}
free(matriz);
return 0;
}

```

5)

```

#include<stdio.h>
#include<stdlib.h>

int* alocação(int N){
    int *p;
    if(N<=0){
        return NULL;
    }else{
        return p=(int *)malloc(N*sizeof(int));
    }
}

void main(){
    int *vetor;
    printf("digite um valor para alocar a memoria de um vetor:\n");
    int n;
    scanf("%d", &n);
    vetor=alocacao(n);
}

```

6)

```

#include<stdio.h>
#include<stdlib.h>
#include <string.h>

char* inverter(char *string, int n){
    char *string_retornado;
    string_retornado=(char *)malloc((n+1)*sizeof(char));
}

```

```

    // verificacao
    if(string_retornado==NULL){
        printf("DEU ERRO NA ALOCACAO!!");
        return 0;
    }
    for(int i=0; i<n; i++){
        string_retornado[i]=string[n-1-i];
    }
    string_retornado[n]='\0';
    return string_retornado;
}

int main(){
    char frase[30];
    printf("digite uma frase:\n");
    setbuf(stdin, NULL);
    fgets(frase, 30, stdin);
    char* frase_retornado= inverter(frase, strlen(frase));
    printf("%s", frase_retornado);
    free(frase_retornado);
    return 0;
}

```

7)
questao repetida.

8)

```

#include<stdio.h>
#include<stdlib.h>

int* somadascolumas(int** matriz, int l, int c){
    int *p=(int *)calloc(c, sizeof(int));

    //verificacao
    if(p==NULL){
        printf("DEU ERRO NA ALOCACAO DA MEMORIA!");
        return 0;
    }

    for(int i =0; i<c; i++){
        for(int m=0; m<l; m++){
            p[i]=p[i]+matriz[m][i];
        }
    }
}

```

```

    }

    }

    return p;
}

void preencher(int **p, int l, int c){
    for(int i=0; i<l; i++){
        for (int m=0;m<c;m++ ){
            printf("digite o valor para linha %d coluna %d", i, m);
            scanf("%d", &p[i][m]);
        }
    }
}

int main(){
    int l, c, **matriz;
    printf("digite um valor para linhas da sua matriz:\n");
    scanf("%d", &l);
    printf("digite um valor para colunas da sua matriz:\n");
    scanf("%d", &c);

    matriz=(int **)malloc(l*sizeof(int*));

    if(matriz==NULL){
        printf("DEU ERRO NA ALOCACAO DA MEMORIA!");
        return 0;
    }

    for(int i=0; i<l; i++){
        matriz[i]=(int *)malloc(c*sizeof(int));

        if(matriz[i]==NULL){
            printf("DEU ERRO NA ALOCACAO DA MEMORIA!");
            return 0;
        }
    }

    preencher(matriz, l ,c);
    int *vetor=somadascolunas(matriz, l, c);

    printf("os valores dos vetores são:\t");
    for(int i=0; i<c; i++){

```

```

        printf("%d\t", vetor[i]);
    }

    for(int i=0; i<l; i++){
        free(matriz[i]);
    }
    free(matriz);
    free(vetor);

    return 0;
}

```

9)

```

#include <stdio.h>
#include <stdlib.h>

float** produtodamatriz(float **matriz1, int l1, int c1, float
**matriz2, int l2, int c2){
    if(c1!=l2){
        return NULL;
    }else{
        float** resultante;
        resultante=(float **)malloc(l1*sizeof(float*));
        if(resultante==NULL){
            printf("Alocacao da memoria deu erro!");
            return NULL;
        }
        for (int i=0; i<l1; i++){
            resultante[i]=(float *)malloc(c2*sizeof(float));

            if(resultante[i]==NULL){
                printf("DEU ERRO NA ALOCACAO DA MEMORIA!");
                return NULL;
            }
        }
        for (int i = 0; i < l1; i++) {
            for (int j = 0; j < c2; j++) {
                resultante[i][j] = 0;
                for (int k = 0; k < c1; k++) {
                    resultante[i][j] += matriz1[i][k] * matriz2[k][j];
                }
            }
        }
    }
}

```

```

        return resultante;
    }
}

void preencher(float **p, int l, int c, int n){
    for(int i=0; i<l; i++){
        for (int m=0;m<c;m++ ){
            printf("digite o valor para linha %d coluna %d da matriz
%d", i, m,n);
            scanf("%f", &p[i][m]);
        }
    }
}

void liberarMatriz(float **matriz, int linhas) {
    if (matriz == NULL) return;
    for (int i = 0; i < linhas; i++) {
        free(matriz[i]);
    }
    free(matriz);
}

int main(){
    int l1,c1, l2, c2;
    printf("digite quantidade de linha para a matriz 1:\n");
    scanf("%d",&l1);
    printf("digite quantidade de coluna para a matriz 1:\n");
    scanf("%d",&c1);
    printf("digite quantidade de linha para a matriz 2:\n");
    scanf("%d",&l2);
    printf("digite quantidade de coluna para a matriz 2:\n");
    scanf("%d",&c2);

    float** matriz1;
    matriz1=(float **)malloc(l1*sizeof(float*));
    if(matriz1==NULL){
        printf("Alocacao da memoria deu erro!");
        return 0;
    }
    for (int i=0; i<l1; i++){
        matriz1[i]=(float *)malloc(c1*sizeof(float));

        if(matriz1[i]==NULL){
            printf("DEU ERRO NA ALOCACAO DA MEMORIA!");

```

```

        return 0;
    }
}

float** matriz2;

matriz2=(float **)malloc(l2*sizeof(float*));
if(matriz2==NULL){
    printf("Alocacao da memoria deu erro!");
    return 0;
}

for (int i=0; i<l2; i++){
    matriz2[i]=(float *)malloc(c2*sizeof(float));

    if(matriz2[i]==NULL){
        printf("DEU ERRO NA ALOCACAO DA MEMORIA!");
        return 0;
    }
}

preencher(matriz1, l1,c1, 1);
preencher(matriz2, l2,c2, 2);
float **vetor=produtodamatriz(matriz1,l1,c1,matriz2,l2,c2);
if(vetor==NULL){
    printf("multiplicação invalida!");
    return 0;
}
printf("\no produto da matriz é:\n");
for (int i=0; i<l1; i++){
    for(int m=0; m<c2; m++){
        printf("%2.f\t", vetor[i][m]);
    }
    printf("\n");
}

liberarMatriz(matriz1, l1);
liberarMatriz(matriz2, l2);
liberarMatriz(vetor, l1);
return 0;
}

```

10)

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

```

```
struct produto{
    int codigo;
    char nome[50];
    int estoque;
    float preco;
};

void cadastro(struct produto *a, int n){
    for (int i=0; i<n; i++){
        printf("digite o codigo do produto %d:\n", i+1);
        scanf("%d", &a[i].codigo);
        printf("digite o nome do produto %d:\n", i+1);
        setbuf(stdin, NULL);
        fgets(a[i].nome, 50, stdin);
        printf("digite a quantidade de estoque dele:\n");
        scanf("%d", &a[i].estoque);
        printf("digite o preco dele:\n");
        scanf("%f", &a[i].preco);
    }
}

void maiorpreco(struct produto *a, int n){
    float maior=0;
    char nometemp[50];
    for(int i=0; i<n; i++){
        if(a[i].preco>maior){
            maior=a[i].preco;
            strcpy(nometemp, a[i].nome);
        }
    }
    printf("o produto: %s tem o maior preco\n", nometemp);
}

void maiorestoque(struct produto *a, int n){
    int maior=0;
    char nometemp[50];
    for(int i=0; i<n; i++){
        if(a[i].estoque>maior){
            maior=a[i].estoque;
            strcpy(nometemp, a[i].nome);
        }
    }
    printf("o produto: %s tem o maior estoque\n", nometemp);
}
```

```

int main(){
    int n;
    printf("digite quantidade de produtos:\n");
    scanf("%d", &n);
    struct produto *coisas;
    coisas=(struct produto *)malloc(n*sizeof(struct produto));
    cadastro(coisas, n);
    maiorestoque(coisas, n);
    maiorpreco(coisas, n);
    free(coisas);
    return 0;
}

```

11)

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int quantidadeigual(int *proprio_numeros, int tamanho1, int
*numeros_sorteados,int tamanho2){
    int qntdd=0;
    for (int i=0; i<6; i++){
        for (int m=0; m<6; m++){
            if (proprio_numeros[i]==numeros_sorteados[m]){
                qntdd++;
            }
        }
    }
    return qntdd;
}

void preencher(int *propriosnumeros, int* numerossorteados){
    for (int i=0; i<6; i++){
        printf("digite o valor %d dos seus numeros\n", i+1);
        scanf("%d", &propriosnumeros[i]);
    }
    for (int i=0; i<6; i++){
        printf("digite o valor %d dos seus numeros sorteados\n", i+1);
        scanf("%d", &numerossorteados[i]);
    }
}

```

```

int* alocarmemoria(int tamanho){
    int *d;
    d=(int *)malloc(tamanho*sizeof(int));
    if(d==NULL){
        printf("ERRO NA ALOCACAO DA MEMORIA!");
        exit(1);
    }
    return d;
}

void acertados(int *proprio_numero, int *sorteados, int *vetor, int
tamanho_vetor){
    int k=0;
    for (int i=0; i<6; i++){
        for (int m=0; m<6; m++){
            if (proprio_numero[i]==sorteados[m]){
                vetor[k]=sorteados[m];
                if(k!=tamanho_vetor){
                    k++;
                }
            }
        }
    }
}

void exibir(int *sorteados, int *corretos, int tamanho){
    printf("os numeros sorteados são:\n");
    for (int i=0; i<6; i++){
        printf("%d\t", sorteados[i]);
    }
    printf("\nos numeros acertados são :\n");
    for (int i=0; i<tamanho; i++){
        printf("%d\t", corretos[i]);
    }
}

int main(){
    int proprios[6], sorteados[6],iguais;
    int *vetor_igual;
    preencher(proprios,sorteados);
    iguais=quantidadeigual(proprios, 6, sorteados,6);
    vetor_igual=alocarmemoria(iguais);
    acertados(proprios, sorteados, vetor_igual, iguais);
}

```

```

    exibir(sorteados, vetor_igual, iguais);
    return 0;
}

```

12)

```

#include <stdio.h>
#include <stdlib.h>

float* alocar(int tamanho){
    float *p;
    p=(float *)calloc(tamanho, sizeof(float));
    if(p==NULL){
        printf("deu erro na alocação!");
        exit(1);
    }
    return p;
}

void menu(){
    printf("digite a sua escolha\n");
    printf("(1)consultar dado em um endereco\n");
    printf("(2)inserir um dado na memoria");
}

int main(){
    int n, e, escolha;
    float valor;
    printf("digite o tamanho de memoria que voce deseja (em bytes)\n");
    scanf("%d",&n);
    n=n/sizeof(float);
    float *p=alocar(n);
    if(p==NULL){
        printf("deu erro na alocação!");
        exit(1);
    }
    menu();
    scanf("%d", &escolha);
    if(escolha<0||escolha>2){
        printf("escolha invalida\n");
        menu();
        scanf("%d", &escolha);
    }
    switch(escolha){
        case 1:
            printf("digite o endereco para consultar:\n (1~%d)", n);

```

```

scanf("%d", &e);
if(e<0||e>n){
    printf("numero invalido!\n");
    return 0;
}
printf("o valor da memoria escolhida é %f", p[e]);
break;

case 2:

printf("digite o local da memoria para preencher valor:\n");
scanf("%d", &e);
if(e<0 ||e>n){
    printf("numero invalido!");
    return 0;
}
printf("qual o valor que deseja preencher:\n");
scanf("%f", &p[e]);
break;

default :
printf("escolha invalida\n");
return 0;
}
free(p);
return 0;
}

```

13)

```

#include <stdio.h>
#include <stdlib.h>

int** alocar( int tamanho1, int tamanho2){
    int **p;
    p=(int **)malloc(tamanho1*sizeof(int*));
    if(p==NULL){
        printf("ERRO NA ALOCACAO DA MEMORIA!");
        exit(1);
    }
    for(int i=0; i<tamanho1;i++){
        p[i]=(int *)malloc(tamanho2*sizeof(int));
        if(p[i]==NULL){

```

```

        printf("ERRO NA ALOCACAO DA MEMORIA!");
        exit(1);
    }
}

return p;
}

void preencher(int **p, int l, int c){
    for(int i=0; i<l; i++){
        for (int m=0;m<c;m++ ){
            printf("digite o valor para linha %d coluna %d", i, m);
            scanf("%d", &p[i][m]);
        }
    }
}

void transposta(int **origem, int linha_o, int coluna_o, int **destino,
int linha_d, int coluna_d){
    for (int i=0; i<linha_o; i++){
        for (int m=0; m<coluna_o; m++){
            destino[m][i]=origem[i][m];
        }
    }
}

void exibir(int**p, int l, int c){
    for(int i=0; i<l; i++){
        for(int m=0; m<c; m++){
            printf("\t%d\t", p[i][m]);
        }
        printf("\n");
    }
}

void top3(int **matriz, int l, int c, int *v1, int *v2, int *v3){
    *v1=0, *v2=0, *v3=0;

    for (int m=0; m<l; m++){
        for(int n=0; n<c; n++){
            if(matriz[m][n]>*v1){
                *v3=*v2;
                *v2=*v1;
                *v1=matriz[m][n];
            }
        }
    }
}

```

```

        }else if(matriz[m][n]>*v2){
            *v3=*v2;
            *v2=matriz[m][n];
        }else if(matriz[m][n]>*v3){
            *v3=matriz[m][n];
        }
    }
}

int main(){
    int linha_a, coluna_a, v1, v2, v3;
    int **matriz, **matriz_inversa;

    printf("digite a quantidade de linhas da sua matriz");
    scanf("%d", &linha_a);
    printf("digite a quantidade de colunas da sua matriz");
    scanf("%d",&coluna_a);

    matriz=alocar(linha_a, coluna_a);
    preencher(matriz, linha_a,coluna_a);

    matriz_inversa=alocar(coluna_a, linha_a);
    transposta(matriz, linha_a, coluna_a, matriz_inversa, coluna_a,
linha_a);
    printf("a matriz digitada é:\n");
    exibir(matriz, linha_a, coluna_a);
    printf("\n a matriz transposta é:\n");
    exibir(matriz_inversa, coluna_a, linha_a);

    top3(matriz, linha_a, coluna_a, &v1, &v2, &v3);
    printf("os maiores tres numeros da matriz sao:\n");
    printf("\t%d\t%d\t%d", v1,v2,v3);

    for (int i = 0; i < linha_a; i++) {
        free(matriz[i]);
    }
    free(matriz);

    for (int i = 0; i < coluna_a; i++) {

```

```
    free(matriz_inversa[i]);
}
free(matriz_inversa);
return 0;
}
```

14)

```
#include <stdio.h>
#include <stdlib.h>

int main() {

    int *vetor;
    int quantidade = 0;
    int capacidade = 10;
    int num;

    vetor = malloc(capacidade * sizeof(int));

    if (vetor == NULL) {
        printf("ERRO NA ALOCACAO!");
        exit(1);
    }

    while (1) {

        printf("Digite o valor:\n");
        scanf("%d", &num);

        if (num == 0)
            break;

        if (quantidade == capacidade) {

            capacidade += 10;

            int *temp = realloc(vetor, capacidade * sizeof(int));

            if (temp == NULL) {
                printf("ERRO NA REALOCACAO!");
                free(vetor);
                exit(1);
            }
        }
    }
}
```

```

        vetor = temp;
    }

    vetor[quantidade] = num;
    quantidade++;
}

printf("\nValores digitados:\n");

for (int i = 0; i < quantidade; i++) {
    printf("%d ", vetor[i]);
}

free(vetor);

return 0;
}

```

15)

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

void menu() {
    printf("\ndigite a sua opcao:\n");
    printf("(1)gravar um nome\n");
    printf("(2)apagar um nome\n");
    printf("(3)procurar um nome e substitui-lo\n");
    printf("(4)procurar um nome e apaga-lo\n");
}

int main() {
    char **matriz;
    int d, e, p;
    printf("digite a quantidade de nomes que queira armazenar:\n");
    scanf("%d", &d);

    matriz = (char **)malloc(d * sizeof(char *));
    if(matriz == NULL) {
        printf("ERRO NA ALOCACAO DA MEMORIA!");
        exit(1);
    }
}

```

```

for (int i = 0; i < d; i++){
    matriz[i] = NULL;
}

menu();
scanf("%d", &e);
switch(e){
    case 1:
        printf("em qual posicao voce quer gravar um
nome?(0~%d):\n", d - 1);
        scanf("%d", &p);

        if (p >= 0 && p < d) {
            if(matriz[p] == NULL){
                matriz[p] = (char *)malloc(30 * sizeof(char));
                if(matriz[p] == NULL){
                    printf("ERRO NA ALOCACAO DA MEMORIA!");
                    exit(1);
                }
                printf("digite o nome para guardar:\n");
                setbuf(stdin, NULL);
                fgets(matriz[p], 30, stdin);
            } else {
                printf("local ja armazenado!\n");
            }
        } else {
            printf("Posicao invalida!\n");
        }
        break;

    case 2:
        printf("digite qual posicao que queira apagar o nome
(0~%d): ", d - 1);
        scanf("%d", &p);

        if (p >= 0 && p < d) {
            if(matriz[p] != NULL){
                free(matriz[p]);
                matriz[p] = NULL;
                printf("nome removido!");
            } else {
                printf("local nao possui nome!");
            }
        }
    }
}

```

```

        }
    } else {
        printf("Posicao invalida!\n");
    }
    break;

case 3: {
    char nome[30], nome_sbs[30];
    int sinal = 0;
    printf("digite o nome que voce quer substituir:\n");
    setbuf(stdin, NULL);
    fgets(nome, 30, stdin);

    for (int i = 0; i < d; i++){
        if(matriz[i] != NULL && strcmp(matriz[i], nome) == 0){
            printf("digite o nome que quer substituir:\n");
            setbuf(stdin, NULL);
            fgets(nome_sbs, 30, stdin);
            strcpy(matriz[i], nome_sbs);
            sinal = 1;
            break;
        }
    }
    if(sinal == 0){
        printf("o nome digitado nao foi encontrado na
matriz\n");
    }
    break;
}

case 4: {
    char nome[30];
    int sinal = 0;
    printf("digite o nome que voce quer apagar: ");
    setbuf(stdin, NULL);
    fgets(nome, 30, stdin);

    for (int i = 0; i < d; i++){
        if(matriz[i] != NULL && strcmp(matriz[i], nome) == 0){
            free(matriz[i]);
            matriz[i] = NULL;
            printf("nome apagado!");
            sinal = 1;
        }
    }
}

```

```

        break;
    }
}
if(sinal == 0){
    printf("o nome digitado nao foi encontrado na
matriz\n");
}
break;
}

default:
    printf("opcao invalida\n");
    break;
}

for(int i = 0; i < d; i++){
    if(matriz[i] != NULL) {
        free(matriz[i]);
    }
}
free(matriz);
return 0;
}

```

16)

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <math.h>

struct info{
    char nome[30];
    float x;
    float y;
};

void imprimir(float **matriz, int d){
    printf("\n");
    for (int i=0; i<d; i++){
        for (int m=0; m<d; m++){
            printf("| %f |", matriz[i][m]);
        }
    }
}

```

```

        printf("\n");
        printf("-----\n");
    }
}

int main(){
    int n;
    struct info *cidade= NULL;
    float **matriz=NULL, distancia;

/*****
***** */

    //alocar memoria por n quantidade de cidade no struct
    printf("digite quantidade de cidades deseja armazenar:\n");
    scanf("%d", &n);
    cidade= (struct info*)malloc(n*sizeof(struct info));
    if(cidade==NULL){
        printf("ERRO NA ALOCACAO DA MEMORIA\n");
        exit(1);
    }

/*****
***** */

    //entrar dados das cidades
    for (int i=0; i<n; i++){
        printf("digite o nome da cidade %d", i+1);
        setbuf(stdin, NULL);
        fgets(cidade[i].nome, 30, stdin);
        printf("digite a posicao x:\n");
        scanf("%f", &cidade[i].x);
        printf("digite a posicao y:\n");
        scanf("%f", &cidade[i].y);
    }

/*****
***** */

    //Criar uma matriz de distancias entre cidades de tamanho N x N;
    matriz=(float**)malloc(n*sizeof(float*));
    if(matriz==NULL){
        printf("ERRO NA ALOCACAO DA MEMORIA!");
        exit(1);
    }

    for (int i=0; i<n; i++){
        matriz[i]=(float*)malloc(n*sizeof(float));

```

```

        if(matriz[i]==NULL) {
            printf("ERRO NA ALOCACAO DA MEMORIA!");
            exit(1);
        }
    }

/*****
***** */

    //Calcular a distância entre cada duas cidades e armazene na
matriz;
    for(int i=0; i<n; i++){
        for(int m=0; m<n; m++){
            distancia=sqrtf(powf(cidade[i].x-cidade[m].x,
2.0)+powf(cidade[i].y-cidade[m].y,2.0));
            matriz[i][m]=distancia;
        }
    }

/*****
***** */

    //imprimir a matriz
    imprimir(matriz, n);

/*****
***** */

    //limpar memoria alocada
    free(cidade);
    for (int i=0; i<n; i++){
        free(matriz[i]);
    }
    free(matriz);
    return 0;
}

```

17)

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>

void ler_aposta(int *vetor, int tamanho){
    int numero_digitado;
    int repetido;

    printf("\n--- ETAPA 1: DIGITE SUA APOSTA ---\n");
    for (int i=0; i<tamanho; i++){

```

```

printf("Digite o valor do numero %d (0 a 100): ", i+1);
scanf("%d",&numero_digitado);

if(numero_digitado<0 || numero_digitado>100){
    printf("[ERRO] Valor fora do limite! Tente
novamente.\n\n");
    i--;
    continue;
}

repetido = 0;
for(int j=0; j<i; j++){
    if(vetor[j] == numero_digitado){
        repetido = 1;
        break;
    }
}

if(repetido == 1){
    printf("[ERRO] Voce ja escolheu esse numero! Escolha
outro.\n\n");
    i--;
} else {
    vetor[i] = numero_digitado;
}
}
}

void sorteia_valores(int *sorteio, int n){
    int numerosorteado;
    int repetido;

    srand(time(NULL));

    for(int i=0; i<n; i++){
        do{
            numerosorteado=rand()%101;
            repetido=0;
            for(int j=0; j<i; j++){
                if(sorteio[j]==numerosorteado){
                    repetido=1;
                    break;
                }
            }
        } while(repetido);
        sorteio[i] = numerosorteado;
    }
}

```

```

        }
        }while(repetido==1);
        sorteio[i]=numerosorteado;
    }
}

int *compara_aposta(int *aposta, int *sorteio, int *qntdd_acertos, int
tamanho){
    int *acertos;
    *qntdd_acertos=0;
    for(int i=0; i<tamanho; i++){
        for (int m=0; m<tamanho; m++){
            if(aposta[i]==sorteio[m]){
                (*qntdd_acertos)++;
            }
        }
    }

    if (*qntdd_acertos == 0) {
        return NULL;
    }

    acertos=(int*)malloc((*qntdd_acertos)*sizeof(int));
    if(acertos==NULL){
        printf("\n[ERRO] Falha grave de memoria!\n");
        exit(1);
    }

    int k=0;
    for(int i=0; i<tamanho; i++){
        for (int m=0; m<tamanho; m++){
            if(aposta[i]==sorteio[m]){
                acertos[k]=aposta[i];
                k++;
            }
        }
    }

    return acertos;
}

int main(){
    int n;

```

```

int *aposta, *sorteio, qntdd_acertos=0, *acertos;

printf("=====\n");
printf("          BINGO DE PROG II          \n");
printf("=====\n\n");

printf("Quantos números você deseja apostar? ");
scanf("%d", &n);

if (n <= 0) {
    printf("\nQuantidade invalida! Encerrando...\n");
    return 1;
}

aposta = (int *)malloc(n*sizeof(int));
if(aposta==NULL){
    printf("\n[ERRO] Falha grave de memoria!\n");
    exit(1);
}
ler_aposta(aposta,n);

sorteio = (int *)malloc(n*sizeof(int));
if(sorteio==NULL){
    printf("\n[ERRO] Falha grave de memoria!\n");
    exit(1);
}
sorteia_valores(sorteio, n);
acertos=compara_aposta(aposta,sorteio, &qntdd_acertos, n);

printf("\n=====\n");
printf("          RESULTADO DO BINGO          \n");
printf("=====\n");

printf("\nNumeros que voce escolheu:\n");
for (int i=0; i<n; i++) {
    printf("[%d] ", aposta[i]);
}

printf("\n\nNumeros sorteados pelo computador:\n");
for (int i=0; i<n; i++) {
    printf("[%d] ", sorteio[i]);
}

```

```

printf("\n\n-----\n");
printf("Quantidade de acertos: %d\n", qntdd_acertos);

if (qntdd_acertos > 0) {
    printf("Numeros acertados: ");
    for (int i=0; i<qntdd_acertos; i++){
        printf("%d ", acertos[i]);
    }
    printf("\n");
} else {
    printf("Infelizmente voce nao acertou nenhum numero desta
vez.\n");
}
printf("-----\n\n");

if (acertos != NULL) {
    free(acertos);
}
free(sorteio);
free(aposta);
return 0;
}

```